

C++ 전처리 파이프라인 단계별 시각화

아동 인물화 원본 이미지가 C++ 전처리 서버를 통과하면서 어떻게 변하는지 확인합니다.
각 단계 옆에 **Why** 섹션을 두어 왜 그 처리가 필요했는지를 함께 기록합니다.

이미지 생성 방법은 [README.md](#) 참조

파이프라인 개요

원본 (임의 크기)

- |
- ▼ [1] ResizeFilter(768)
- ▼ [2] ColorValidationFilter ← 컬러 이미지면 422 즉시 반환
- ▼ [3] DenoiseFilter
- ▼ [4] HybridPreprocessFilter
 - ├ 4a. Grayscale 변환
 - ├ 4b. 적응형 이진화 (AdaptiveThreshold)
 - ├ 4c. 형태학 연산 (MORPH_CLOSE)
 - ├ 4d. Smart Crop (ROI 감지)
 - ├ 4e. Letterbox Resize (512×512)
 - └ 4f. 3채널 병합 (R/G/B)
- |
- ▼ 512×512×3 RGB 이미지 → Python AI 서버

코드 위치: `preprocess-server/src/core/pipeline_factory.cpp`

Stage 00: 원본 입력

이미지

이미지



- 형식: BGR, 임의 크기
- 특징: 사용자가 업로드한 그대로. 해상도, 비율, 밝기 모두 불규칙

Stage 01: ResizeFilter (768×768)

입력

출력

입력

출력



- **파라미터:** `targetSize=768, withPadding=false`
- **동작:** 장축이 768이 되도록 비율 유지 리사이즈 (패딩 없음)

Why: 이후 모든 픽셀 연산(이진화, 형태학, 거리 변환)을 고정 해상도에서 실행해 레이턴시를 **47% 감소** (183ms → 97ms). 768은 이후 512×512 letterbox 품질과 속도의 균형점.

코드: `preprocess-server/src/filters/resize_filter.cpp`

ADR: `docs/standards/ADR-parameter-rationale.md`

Stage 02: DenoiseFilter

입력

출력

입력

출력



- **파라미터:** `gaussianSize=5`, `medianSize=0` (비활성화)
- **동작:** Gaussian Blur 5×5로 고주파 노이즈 제거

Why: 이진화(AdaptiveThreshold)는 노이즈에 민감해서, 사전에 스무딩하지 않으면 연필 질감의 미세한 텍스처가 선(stroke)으로 오인되어 불필요한 점들이 생긴다. Median Blur는 비활성화 — 인물화의 얇은 선을 뭉갤 수 있어 제외.

코드: `preprocess-server/src/filters/denoise_filter.cpp`

Stage 03a: Grayscale 변환

입력

출력



- **동작:** BGR → GRAY (`cv::COLOR_BGR2GRAY`)

Why: 이진화와 형태학 연산은 단채널(1D) 강도값만 필요.

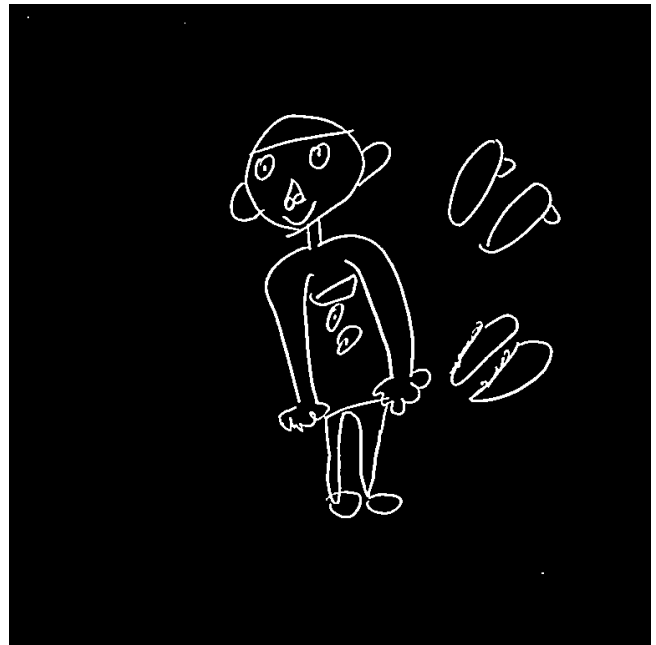
컬러 정보는 ColorValidationFilter에서 이미 사전 검증했으므로 이 시점에는 불필요.

Stage 03b: 적응형 이진화

입력



출력



- **파라미터:** `AdaptiveThreshold`, `BINARY_INV`, `blockSize=11`, `C=2`
- **동작:** 국소 영역별 임계값으로 이진화. 선 → 흰색, 배경 → 검은색

Why: 전역 임계값(Otsu)은 종이 밝기가 균일하지 않을 때 한쪽 영역을 통째로 날릴 수 있다.

`Adaptive`는 11×11 블록 단위로 임계값을 계산해, 조명 불균일이나 흐린 연필선 모두 포착.

`BINARY_INV`는 이후 ROI 감지를 위해 선이 흰색(=전경)이어야 하기 때문.

코드: `preprocess-server/src/filters/binarize_filter.cpp`

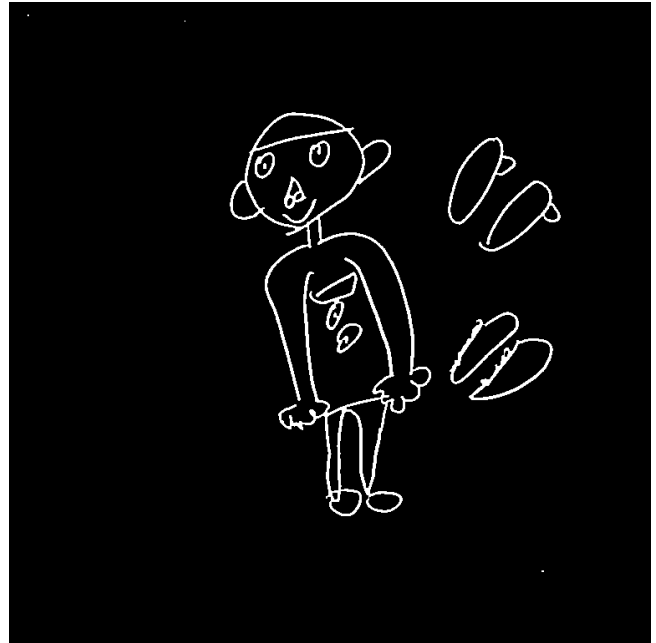
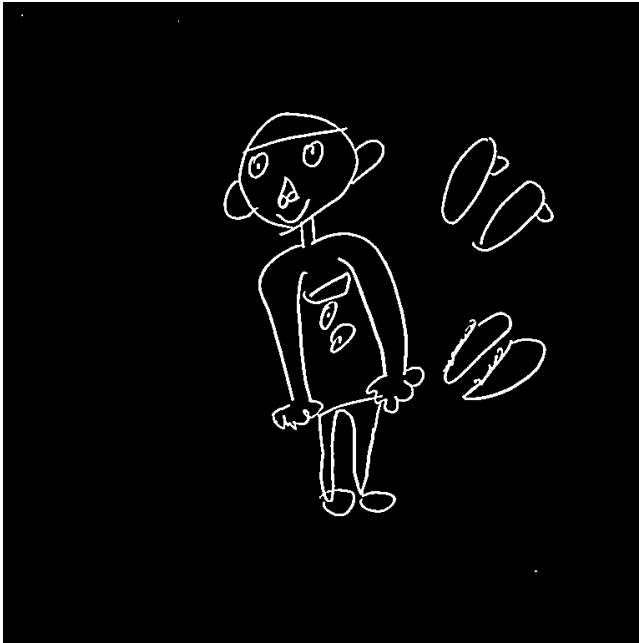
Stage 03c: 형태학 연산 (MORPH_CLOSE)

입력

출력

입력

출력



- **파라미터:** MORPH_CLOSE, kernel 3x3
- **동작:** Dilation → Erosion 순서로 끊어진 선을 연결하고 작은 구멍 메움

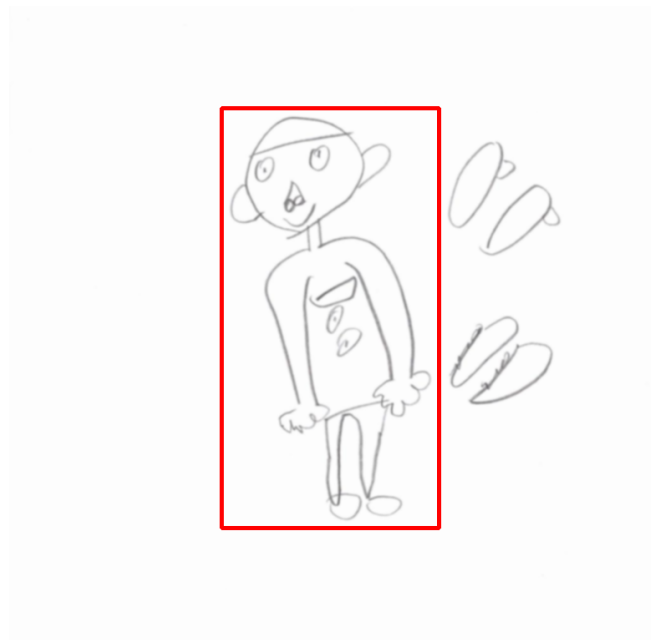
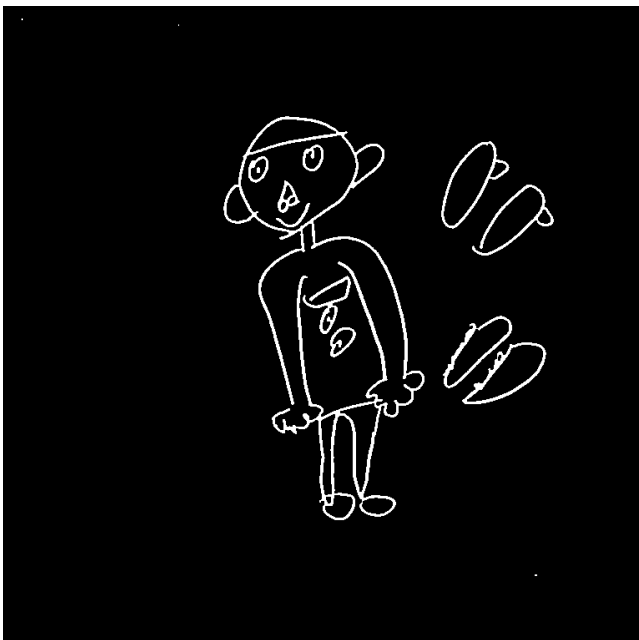
Why: 연필화는 선이 불연속적으로 그어지는 경우가 많다. 끊어진 선은 컨투어 감지에서 하나의 덩어리로 인식되지 않아 ROI 계산이 부정확해진다. MORPH_CLOSE가 작은 간격을 메워 메인 인물을 하나의 connected component로 만들어 준다.

코드: `preprocess-server/src/filters/morphology_filter.cpp`

Stage 03d: Smart Crop (ROI 감지)

입력

출력



- **동작:** 컨투어 감지 → 면적 상위 dominant 컨투어의 bounding box 추출 (+ padding 10px)

- **빨간 박스:** 감지된 ROI 영역

Why: 배경 여백을 포함한 채로 512×512에 맞추면 인물이 작아져 특징 추출 품질이 떨어진다.
메인 인물 영역만 잘라내서 리사이즈하면, 같은 512×512 안에 인물이 꽉 차게 들어가
EfficientNet이 더 풍부한 정보를 학습/추론할 수 있다.

코드: `preprocess-server/src/filters/hybrid_preprocess_filter.cpp` — `GetContentROI()`

Stage 03e: Letterbox Resize (512×512)

입력 (ROI 영역)	출력
	

- **동작:** ROI 영역을 비율 유지 리사이즈 후 흰색(255) 캔버스 중앙에 배치

Why: AI 모델은 고정 입력 크기를 요구한다. 단순 stretch(비율 무시 리사이즈)를 하면
인물의 사지 비율이 왜곡되어 HFD 문항 판정(팔 길이, 다리 비율 등)에 오류가 발생한다.
Letterbox는 비율을 보존하면서 빈 공간만 흰색으로 채운다.

코드: `preprocess-server/src/filters/hybrid_preprocess_filter.cpp` — `PrepareLetterbox()`

Stage 04: 3채널 병합 결과

3채널 병합 후 각 채널을 분리해서 보면 각자 다른 정보를 담고 있음을 확인할 수 있다.

R 채널 — Grayscale (명도 / 필압 정보)

이미지

이미지



- 원본 그레이스케일 강도를 그대로 보존
- 연필 압력(진하게 그린 부분 vs 연하게 그린 부분)이 명암으로 표현됨

G 채널 — Inverted Binary (선 형태 / 윤곽)

이미지

이미지



- 이진화를 반전: 선 → 흰색(255), 배경 → 검은색(0)
- 인물의 외곽선과 내부 특징선을 명확하게 분리

B 채널 — Distance Map (선 굵기 / 거리 정보)

이미지

이미지



- 각 픽셀에서 가장 가까운 "배경(0)" 픽셀까지의 거리를 정규화
- 선이 굵을수록 선 중심부의 값이 크게 나타남 → 선 굵기 정보 내포

최종 3채널 병합 (AI 서버 입력)

이미지

이미지



- **R**: 명도 (필압), **G**: 윤곽 (형태), **B**: 거리 (굵기)
- 단순 그레이스케일 1채널보다 3배 풍부한 정보를 EfficientNet에 제공
- 크기: **512×512×3**, dtype: uint8

Why 3채널인가? EfficientNet-B2는 3채널 입력을 기대한다(ImageNet 사전학습). 동일한 이미지를 단순 복제(`cv::cvtColor` → `repeat`)해서 3채널로 만들 수도 있지만, 각 채널에 **보완적인 정보**를 담으면 모델이 더 다양한 특징을 학습할 수 있다.

코드: `preprocess-server/src/filters/hybrid_preprocess_filter.cpp` — `ConstructChannels()`
ADR: `docs/reference/tech-references/AI/hybrid_3channel_rationale.md`

정리 — 단계별 입출력 요약

Stage	필터	입력	출력	핵심 이유
00	—	원본	BGR 임의 크기	—
01	ResizeFilter	BGR 임의	BGR ≤768px	47% 레이턴시 감소
02	DenoiseFilter	BGR 768	BGR 768 (smooth)	이진화 노이즈 방지
03a	cvtColor	BGR 768	Gray 768	단채널 연산 준비

Stage	필터	입력	출력	핵심 이유
03b	BinarizeFilter	Gray	Binary (INV)	선/배경 분리
03c	MorphologyFilter	Binary	Binary (closed)	끊어진 선 연결
03d	GetContentROI	Binary	ROI rect	인물 영역 추출
03e	Letterbox	Gray ROI	Gray 512	비율 보존 리사이즈
04	ConstructChannels	Gray+Binary	RGB 512×512×3	다채널 정보 통합